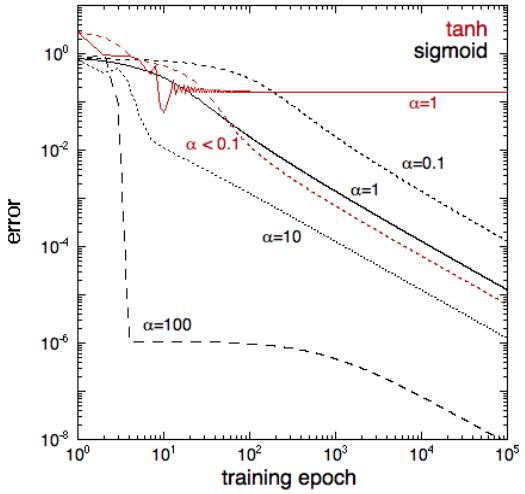




PHYS250 HOMEWORK #3 (due Nov. 15)

On your own (no help from Claude, other than for debugging):

* Build a single-layer perceptron as described in class, profiting from the matrix algebra and matrix manipulation commands of FORTRAN 90. Involve the commands MATMUL, SUM, TRANSPOSE. Train it in batch mode with the four examples mentioned in our slides, and test it with the other four remaining cases. Does it work? If you observe any limitations, explain their cause. Does it work for the example provided in the slides involving the XOR function?

* Compare the performance of the tanh and sigmoid transfer functions in your code (do not forget to change their derivative when doing this, also in the validation process). Is one faster than the other? (use the command CALL CPU_TIME( ) to time your code at given intervals). Does one work better than the other for the test inputs? If so, why?

* Optimize the learning rate separately for these two transfer functions. Do this by plotting the error vs. training epoch, as in the example above. Describe your observations.

* Does the performance (error vs. epoch, or error in validation set) improve if you expand the training set? (e.g., by adding one more case to it). If you scramble it? (stay at four training cases, but use other different cases for training). Do not forget to update the dimensions of your matrices when adding or subtracting new cases to the training set.

* Once your code is built, test it for other hidden patterns of your own that link inputs and outputs: in doing so, develop a feeling for the limitations of the single neuron perceptron. Comment on your observations.

Vibe-coding your way into the Skynet IPO:

* Find the most economical (i.e., simplest) NN architecture able to successfully describe the XOR relation for the three inputs used for our perceptron. Explain what made the difference.

* Find the most economical (i.e., simplest) NN architecture able to deal with the MNIST character recognition dataset downloadable at https://archive.ics.uci.edu/ml/datasets/Letter+Recognition. Test and describe the architecture and its performance.